CONTENTS

Advanced | Vector search infrastructure | IBM Technology

Vector Databases: Storing Meaning Instead of Fields

Understand embeddings, distance metrics and the approximate-nearest-neighbour indexes that make semantic search fast.

For engineers who need to choose, tune or debug a vector store rather than just call one · 26 July 2026

Watch the source video | Read the condensed version | Read the highlights

BEFORE YOU START

- What a relational database does
- Comfort reading Python
- Knowing what a vector is — a list of numbers with a position in space

Overview

A relational database can store an image perfectly well — the binary blob, the file format, the creation date, whatever tags somebody typed in. What it cannot do is answer 'find me pictures with a similar colour palette' or 'find landscapes with mountains in the background', because those concepts are not in any column. That mismatch between how computers store data and how humans understand it is called the semantic gap.

Vector databases close it by storing meaning as geometry. Every item becomes an array of numbers positioned so that similar things sit close together, and 'find similar' becomes a mathematical operation over those positions rather than a predicate over fields.

The engineering interest is mostly in what happens after that idea. Comparing a query against millions of thousand-dimensional vectors exhaustively is too slow, so real systems use approximate nearest-neighbour indexes that trade a small amount of accuracy for a very large amount of speed. Understanding how HNSW and IVF make that trade — and what you give up — is the difference between using a vector database and being able to fix one.

KEY TAKEAWAYS

- ✓ The semantic gap is the mismatch between structured storage and conceptual similarity — SQL predicates cannot express 'looks like this'.
- ✓ An embedding is an array of numbers positioned so that semantically similar items land close together and dissimilar ones land far apart.
- ✓ Individual dimensions in real embeddings are almost never interpretable; the geometry as a whole carries the meaning.
- ✓ Each modality has its own embedding models, and vectors from different models are never comparable.
- ✓ Embedding models extract progressively more abstract features layer by layer — edges to objects, words to meaning.
- ✓ Exhaustive search is O(N·d) and becomes impossible at scale, which is why vector indexes exist.
- ✓ HNSW navigates a multi-layer proximity graph; IVF partitions the space into clusters and searches only the nearest few.
- ✓ ANN indexes trade a small, measurable amount of recall for very large speed gains — and that recall must actually be measured.

01 The semantic gap

0:00

> You can store an image in a relational database along with its metadata and tags, and still be unable to ask the questions that matter about it.

Take a photograph of a sunset over a mountain vista and put it in a relational database. You can store the binary image data. You can store basic metadata — file format, creation date, dimensions. You can add manually curated tags: sunset, landscape, orange.

That gives you a basic retrieval mechanism, and it misses the image's semantic content almost entirely. How would you query for images with a similar colour palette using those fields? Or landscapes with mountains in the background? Those concepts are not represented anywhere in the structured columns, and no amount of SQL will recover them.

That disconnect between how computers store data and how humans understand it is the semantic gap. A query like `SELECT * WHERE color = 'orange'` falls short because it cannot capture the nuanced, multi-dimensional nature of unstructured data. Every attribute has to be anticipated, named and populated in advance by a person — which means the questions you can ask were fixed before anyone knew what they would want to ask.

Tagging looks like a solution and is not. Tags are discrete where similarity is continuous, so there is no such thing as 'a bit sunset'. They are incomplete, because nobody tags for a property until someone needs it. And they are inconsistent, because two people tagging the same image will not agree. The gap is

structural rather than a matter of effort.

KEY POINTS

- → Relational storage handles the file and its metadata but not its content.
- → Manual tags fix the queryable attributes before anyone knows what will be asked.
- → Similarity is continuous; tags are discrete, so degree cannot be expressed.
- → Tagging is incomplete and inconsistent at any realistic scale.
- → The semantic gap is structural — it cannot be closed by tagging harder.

What the relational row actually holds

Everything here is about the file rather than the picture. Nothing in the row would let you find this image by starting from a different image that looks like it.

```
CREATE TABLE images (
    id          SERIAL PRIMARY KEY,
    data        BYTEA,          -- the actual pixels, opaque to the database
    format      TEXT,           -- 'jpeg'
    created_at  TIMESTAMP,
    tags        TEXT[]          -- {'sunset','landscape','orange'}
);

-- answerable:   SELECT * FROM images WHERE 'sunset' = ANY(tags);
-- not answerable: "images with a similar colour palette to image 42"
--                 "landscapes with mountains in the background"
--                 "something with this mood"
```

Why more tags do not fix it

The failure is not that the tag vocabulary is too small. It is that similarity is a relation between items, and tags are properties of one item — so the question 'what is like this?' has no expression in the schema at all.

```
tags are discrete      an image is 'sunset' or not; never 0.83 sunset
tags are incomplete    nobody tagged for 'elevation' until someone asked
tags are inconsistent  'orange' / 'warm' / 'golden' — three taggers, three words
tags are unary         no way to express "close to image 42"
```

02 Vectors as the representation

2:11

> Represent each item as an array of numbers positioned so that similar items are near one another, and similarity search becomes arithmetic.

A vector database represents data as mathematical vector embeddings — essentially arrays of numbers. These vectors capture the semantic essence of the data: similar items are positioned close together in the vector space, and dissimilar items far apart.

Once that holds, similarity search becomes a mathematical operation. Finding vector embeddings that are close to one another translates directly to finding semantically similar content. There is no vocabulary to agree on and no attribute to anticipate — the question 'what is like this?' becomes 'what is nearby?', which is a well-defined geometric query.

All kinds of unstructured data can live in this space. Images like the mountain sunset, text files, audio. The complex object is transformed into a vector embedding, and the embedding is what the database stores and indexes.

Worth being precise about one thing: 'close' requires a distance metric, and the choice is not arbitrary. Cosine similarity measures the angle between vectors and ignores their magnitude, which is what you usually want for text — a long document and a short phrase expressing the same idea point the same way. Euclidean distance measures straight-line separation and is sensitive to magnitude. Dot product combines both, rewarding alignment and length. Most text embedding models are trained with cosine in mind, and using the wrong metric quietly degrades results without producing any error.

KEY POINTS

- → An embedding is an array of numbers whose position encodes semantic content.
- → Similar items sit close together; dissimilar ones sit far apart.
- → 'What is similar?' becomes 'what is nearby?', a well-defined geometric query.
- → Images, text and audio all become vectors and share the same machinery.
- → The distance metric matters: cosine ignores magnitude, Euclidean does not, dot product mixes both.
- → Using a metric the embedding model was not trained for degrades results silently.

The three metrics, and when each applies

If your vectors are L2-normalised, cosine and dot product rank identically and Euclidean produces the same ordering too — which is why many stores normalise on insert and stop worrying about it. If they are not normalised, the three can disagree substantially.

```
import numpy as np

def cosine(a, b):      # angle only — magnitude ignored
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def dot(a, b):         # angle and magnitude together
    return np.dot(a, b)

def euclidean(a, b):   # straight-line distance; smaller is closer
    return np.linalg.norm(a - b)


# after L2 normalisation all three rank identically:
def normalise(v):
    return v / np.linalg.norm(v)

# check what your embedding model was trained for before choosing —
# the wrong metric produces plausible, quietly worse results
```

The interface a vector store exposes

Two operations carry the whole model. Note that `search` takes a vector rather than text: embedding happens in your application, which is what makes it your responsibility to use the same model on both sides.

```
store.upsert(id="img-42", vector=embed(image), metadata={"format": "jpeg"})

hits = store.search(vector=embed(query_image), k=10)
# → [(id, distance, metadata), ...] ordered by proximity

# there is no WHERE clause describing what "similar" means.
# similarity is defined by the embedding model, not by the query.
```

03 What a vector actually encodes

3:38

> Each position represents a learned feature. In a toy example those features are readable; in real systems they are not.

An embedding is an array of numbers where each position represents some learned feature. Take a simplified example of the mountain picture, with a vector beginning 0.91, 0.15, 0.83.

Read that as: 0.91 in the first dimension indicates significant elevation changes, because these are mountains. 0.15 in the second shows few urban elements — there are no buildings in the frame, so the score is low. 0.83 in the third represents strong warm colours, the sunset. And so on through however many dimensions the model produces.

Two caveats have to travel with that example, and they matter more than the example does. Real embeddings contain hundreds or thousands of dimensions, not three. And individual dimensions rarely correspond to anything so cleanly interpretable — meaning is distributed across the whole vector rather than allocated one concept per axis.

That second point has practical consequences people trip over. You cannot inspect dimension 47 to find out what the model thinks about elevation. You cannot build a filter by thresholding a single coordinate. And you cannot debug an embedding by reading it — the only meaningful operations are comparisons between whole vectors. If you need a queryable attribute, extract it explicitly and store it as metadata alongside the vector.

Dimensionality itself is a real trade. More dimensions capture finer distinctions and cost more memory, more bandwidth and more time in every distance computation. A million vectors at 1536 dimensions in float32 is around 6 GB before any index overhead, which is why quantisation exists.

KEY POINTS

- → Each position in the vector corresponds to a learned feature.
- → Real embeddings have hundreds or thousands of dimensions.
- → Individual dimensions are almost never interpretable — meaning is distributed.
- → You cannot filter or debug by inspecting single coordinates; only whole-vector comparisons are meaningful.
- → Anything you need to filter on must be stored explicitly as metadata.
- → Dimensionality trades expressiveness against memory, bandwidth and query time.

The toy vector versus the real one

The left column is a teaching device. The right is what you will actually get back, and no coordinate in it has a name.

```
toy (interpretable, 3 dims)        real (1536 dims, uninterpretable)
──────────────────────────────────────────────────────────────────────

0.91  elevation change              [−0.0231,  0.0417, −0.0088,
0.15  urban elements                  0.0192, −0.0364,  0.0521,
0.83  warm colour                     0.0077,  0.0143, −0.0296,
                                      ... 1527 more ... ]

# no dimension means "elevation". the concept is spread across many
# coordinates, and no single one can be read, thresholded or filtered on.
```

Sizing a vector index before you build it

Run this before choosing an embedding dimension. Memory is usually the binding constraint on vector search, and the graph index adds substantially on top of the raw vectors.

```
def raw_bytes(n_vectors, dims, dtype_bytes=4):      # float32
    return n_vectors * dims * dtype_bytes

for dims in (384, 768, 1536, 3072):
    gb = raw_bytes(1_000_000, dims) / 1e9
    print(f"{dims:>5} dims  {gb:>6.2f} GB per million vectors")

#   384 dims    1.54 GB
#   768 dims    3.07 GB
#  1536 dims    6.14 GB
#  3072 dims   12.29 GB

# an HNSW graph adds roughly M x 8 bytes per vector on top of this
```

04 Comparing two vectors

4:20

> Two sunsets, one mountain and one beach: similar where they share meaning, different where they do not.

Put a second image next to the first — a sunset at the beach — and look at its vector: 0.12, 0.08, 0.89.

The third dimension is 0.83 for the mountain and 0.89 for the beach, which is very close. Both are sunsets, both have strong warm colours. The first dimension differs sharply, 0.91 against 0.12, because a beach has minimal elevation change compared to mountains.

This is the whole mechanism of semantic search, visible in three numbers. The two images share some meaning and differ in other meaning, and the vector expresses both facts simultaneously, per dimension. A tag system would have to decide whether these two images are 'the same' or 'different'; the vector says they are alike in one respect and unlike in another, and the distance measure aggregates that into a single number.

It also shows why the query determines what counts as similar. Search with a warm sunset palette and both images are near neighbours. Search with a mountain landscape and they separate. The database is not storing a fixed notion of similarity — it stores positions, and each query cuts through that space from a different direction.

KEY POINTS

- → Two items can be alike in some dimensions and unlike in others simultaneously.
- → The distance metric aggregates per-dimension agreement into one score.
- → A tag system must call two items same or different; a vector expresses degree.
- → What counts as similar depends on where the query sits, not on stored labels.
- → This is why semantic search generalises to questions nobody anticipated.

Two sunsets, dimension by dimension

Aggregate distance hides this structure — you see one number and not which dimensions produced it. Being able to picture the per-dimension view is what makes similarity results interpretable when they surprise you.

```
                 mountain   beach    agreement
──────────────────────────────────────────────
elevation          0.91      0.12     far apart
urban elements     0.15      0.08     both low
warm colour        0.83      0.89     nearly equal

# similar overall — the sunset dominates — but clearly distinguishable
# on the axis that separates a mountain from a shoreline
```

The query decides what similar means

The same two vectors, two different queries, two different verdicts. No stored attribute changed — only the direction from which the space was queried.

```
q1 = [0.10, 0.10, 0.90]      # "a warm sunset"
  → mountain 0.87, beach 0.99     both close: the query is about colour

q2 = [0.95, 0.10, 0.20]      # "a mountain landscape"
  → mountain 0.93, beach 0.24     they separate: the query is about terrain
```

05 Where embeddings come from

5:48

> Embedding models trained on massive datasets, with each layer extracting progressively more abstract features.

Embeddings are produced by embedding models trained on massive datasets, and each type of data has its own specialised model. CLIP is a common choice for images. For text you might use GloVe. For audio, Wav2vec.

The process is similar across modalities. Data passes through multiple layers, and each layer extracts progressively more abstract features. For images the early layers detect basic structure like edges; deeper layers recognise entire objects. For text the early layers work at the level of individual words, while deeper layers capture context and meaning. The embedding is taken from one of those deeper layers, where the high-dimensional vector captures the essential characteristics of the input.

One distinction is worth adding, because it separates older text embeddings from what you would reach for today. Static word embeddings such as GloVe assign each word one fixed vector regardless of context, so 'bank' has a single representation whether the sentence is about rivers or money. Modern transformer-based sentence encoders produce a representation that depends on the surrounding text, so the same word embeds differently in different sentences — and a whole passage embeds as a unit rather than as an average of its words. For retrieval over documents this matters a great deal.

The operational rule that follows from all of this: vectors are only comparable within the model that produced them. Embed your corpus with one model and your queries with another and you get numbers, distances, and results that are entirely meaningless. Changing embedding model means re-embedding everything — which makes it a migration, not a configuration change.

KEY POINTS

- → Different modalities use different embedding models: CLIP for images, Wav2vec for audio, text encoders for text.
- → Layers extract progressively more abstract features — edges to objects, words to meaning.
- → The embedding is read from a deep layer that captures essential characteristics.
- → Static word embeddings give one vector per word; contextual encoders vary by surrounding text.
- → Vectors are only comparable within the same model — mixing models produces meaningless results.
- → Changing embedding model requires re-embedding the entire corpus.

Abstraction increasing with depth

The reason the embedding comes from a deep layer rather than an early one. Early activations describe the surface of the input; deep activations describe what it is.

```
images                          text
──────────────────────────────────────────────────────
layer 1   edges, gradients      tokens, subwords
layer 2   textures, corners     local syntax
   ...                             ...
layer n   objects, scenes       context, meaning, intent
              ↓                       ↓
        embedding vector          embedding vector
```

Static versus contextual text embeddings

The practical consequence for retrieval. A static model cannot distinguish these two sentences at the word level; a contextual encoder places them in different regions, which is exactly what a search over mixed-domain documents needs.

```
"I sat on the river bank"      "I deposited it at the bank"

GloVe (static)     : "bank" → the same vector in both sentences
transformer encoder: "bank" → two clearly different vectors

# and a sentence encoder embeds the whole passage as one unit,
# rather than averaging independent word vectors
```

The mixed-model bug

Nothing errors. The dimensions match, the distances compute, results come back ranked. They are noise. Store the model name and version alongside the index and assert on it at query time — this bug is otherwise very hard to see.

```
# indexed months ago
store.upsert(id=doc.id, vector=model_a.embed(doc.text))

# queried today, after someone changed a default
hits = store.search(model_b.embed(question), k=5)

# same dimensionality, valid distances, plausible-looking results,
# and no relationship whatsoever to relevance.

assert store.meta["embedding_model"] == model.name, "index/query mismatch"
```

06 Similarity search, and why brute force runs out

7:16

> Comparing a query against every vector is exact and simple, and becomes impossible somewhere in the millions.

With embeddings in place you can perform operations that were not possible with a relational database — most importantly similarity search, finding the items closest to a query by locating the nearest vectors in the space.

The obvious implementation is to compare the query against every vector and keep the best few. This is exact: it returns the true nearest neighbours by definition. For small collections it is also the correct choice, and it is worth remembering that a few hundred thousand vectors is genuinely fine on modern hardware with a vectorised dot product.

But the cost is O(N·d) — every vector, every dimension. When you have millions of vectors of hundreds or thousands of dimensions each, comparing against every one simply becomes too slow. And it is not only compute: an exhaustive scan reads the entire index from memory on every query, so you are bounded by memory bandwidth as much as by arithmetic.

That is what motivates vector indexing. The insight behind every ANN algorithm is the same: you rarely need the exact nearest neighbours. You need results that are almost always the nearest, returned in milliseconds instead of seconds. Giving up a small, measurable amount of accuracy buys orders of magnitude of speed — and 'measurable' is the load-bearing word, because that trade is only sound if you know where you sit on it.

KEY POINTS

- → Exhaustive search is exact and simple, and correct for small collections.
- → Its cost is O(N·d) per query — every vector, every dimension.
- → At millions of vectors it is bounded by memory bandwidth as well as arithmetic.
- → ANN indexes give up exactness for orders-of-magnitude speed.
- → The trade is only sound if the lost recall is actually measured.

Exhaustive search, and where it stops working

The vectorised form is much faster than a Python loop and still linear in the corpus. Somewhere between a hundred thousand and a few million vectors — depending on dimensions, hardware and your latency budget — it stops meeting a real-time requirement.

```
import numpy as np

def exact_search(query, matrix, k=10):
    # matrix: (N, d), L2-normalised so dot product == cosine
    scores = matrix @ query                 # (N,) — every vector, every dim
    top = np.argpartition(-scores, k)[:k]   # cheaper than a full sort
    return top[np.argsort(-scores[top])]

# 1M vectors x 1536 dims x 4 bytes = 6.1 GB read per query
# correctness: perfect.   latency: not interactive.
```

The bargain every ANN index offers

Stated plainly, because the numbers are dramatic and the cost is real. Missing one true neighbour out of ten usually does not change an answer; waiting two seconds always changes the product.

```
exact   : returns the true top-10, every time.       seconds.
ANN     : returns 9 or 10 of the true top-10.        milliseconds.

# the question is never "is approximation acceptable?" in the abstract.
# it is "what recall do I get at the latency I need?" — a number you
# measure against exact search on your own data.
```

07 ANN indexing: HNSW and IVF

8:00

> Two ways to avoid looking at everything — navigate a proximity graph, or search only the nearest clusters.

Vector indexing uses approximate nearest neighbour algorithms. Instead of finding the exact closest match, these quickly find vectors very likely to be among the closest.

HNSW — Hierarchical Navigable Small World — builds multi-layered graphs connecting similar vectors. The top layer is sparse, with long-range links; each layer down is denser. A search enters at the top, greedily walks toward the query through the sparse layer until it cannot improve, drops a level, and repeats. The upper layers act like an express network that covers distance quickly; the bottom layer does the fine-grained work. Three parameters control it: `M`, how many neighbours each node keeps, which drives memory; `ef_construction`, how thoroughly the graph is built, which drives index build time and quality; and `ef_search`, how wide the search beam is at query time, which is the dial you turn to trade recall against latency without rebuilding anything.

IVF — Inverted File Index — takes a different route. It divides the vector space into clusters, typically by running k-means to find `nlist` centroids, and assigns each vector to its nearest one. At query time it compares the query against the centroids, picks the closest `nprobe` clusters, and searches only inside those. Set `nprobe` to 1 and it is very fast and misses anything sitting just across a boundary; raise it and recall climbs toward exact as latency rises.

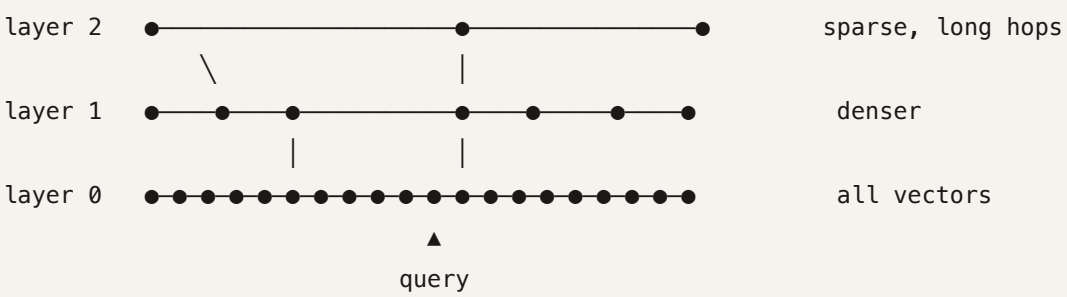
The practical differences: HNSW generally gives better recall at a given latency and uses substantially more memory, and it handles incremental inserts naturally. IVF is much more memory-efficient and pairs well with quantisation for very large collections, but needs a training pass over representative data and degrades if the distribution shifts after training. Both trade a small amount of accuracy for large improvements in search speed.

KEY POINTS

- → HNSW builds a layered proximity graph and greedily navigates from sparse to dense layers.
- → HNSW parameters: M for connectivity and memory, ef_construction for build quality, ef_search for the recall/latency dial at query time.
- → IVF clusters the space with k-means and searches only the nearest nprobe clusters.
- → IVF needs a training pass and degrades if the data distribution shifts afterwards.
- → HNSW gives better recall per millisecond and costs more memory; IVF is leaner and quantises well.
- → Both trade a small amount of accuracy for large speed gains.

How an HNSW search moves

The upper layers exist purely to cover distance quickly. Without them the greedy walk on the dense bottom layer would take many more hops and could get stuck in a local minimum far from the query.




```
# enter at the top, greedily step toward the query, descend when no
# neighbour improves, repeat. the bottom layer does the fine work.
```

Tuning the recall/latency dial

`ef_search` is changeable per query with no rebuild, which makes it the right knob for balancing an interactive path against a batch path in the same system. Run this against exact search on your own data — the shape of the curve is dataset-specific.

```
index = hnswlib.Index(space="cosine", dim=1536)
index.init_index(max_elements=1_000_000, M=32, ef_construction=200)
index.add_items(vectors, ids)

for ef in (16, 32, 64, 128, 256, 512):
    index.set_ef(ef)                      # query-time only, no rebuild
    labels, _ = index.knn_query(queries, k=10)
    print(f"ef_search={ef:<4} recall@10={recall(labels, ground_truth):.3f}")

# ground_truth comes from exact search over the same vectors.
# without it you have no idea what your index is losing.
```

IVF, and the boundary problem

The failure mode is specific and worth picturing: a vector sitting just the wrong side of a cluster boundary is invisible at nprobe=1 no matter how close it is to the query. Raising nprobe is how you buy it back.

```
nlist  = 4096      # clusters, from k-means over a training sample
nprobe = 16        # how many to actually search

#   query ●
#         ┆ closest centroid → searched
#    ┌────┼────┐  ┌──────────┐
#    │    ●    │  │  ○ ←──── true nearest neighbour, just across the line
#    └─────────┘  └──────────┘
#                  not searched at nprobe=1

# nprobe=1     fastest, lowest recall
# nprobe=nlist equivalent to exhaustive search
# IVF also needs index.train(sample) before adding vectors
```

Filtering plus ANN: the trap

Metadata filters and approximate search interact badly, and this surprises people in production. Post-filtering can return fewer results than requested — or none — when the filter is selective. Use a store with native filtered search, or over-fetch deliberately.

```
# post-filter: search 10, then discard non-matches → often returns 2
hits = [h for h in store.search(q, k=10) if h.meta["lang"] == "es"]

# over-fetch: crude but works when the filter is not too selective
hits = [h for h in store.search(q, k=200) if h.meta["lang"] == "es"][:10]

# best: let the index apply the predicate during graph traversal
hits = store.search(q, k=10, filter={"lang": "es"})
```

08 Vector databases inside RAG

8:41

> Store document chunks as embeddings, find the relevant ones by similarity, hand them to a language model.

Vector databases are a core component of retrieval augmented generation. Chunks of documents, articles and knowledge bases are stored as embeddings. When a user asks a question, the system finds the relevant text chunks by comparing vector similarity, and feeds those chunks to a language model, which generates a response using the retrieved information.

So a vector database is two things at once: a place to store unstructured data, and a place to retrieve it quickly and semantically. In a RAG system it is the second role that dominates — the source documents usually live somewhere else, and the vector store holds chunks and vectors in service of retrieval.

Everything in this lesson shows up as a knob in that system, which is why the internals are worth knowing. Chunk size determines what a single vector has to represent. The embedding model determines what counts as similar, and cannot be changed without re-embedding everything. The distance metric has to match how that model was trained. And the ANN index parameters determine how much of the true top-k you actually receive — which is a ceiling on the whole application's accuracy, sitting one layer below anything you can see in a prompt.

That last one is the point most worth carrying away. When a RAG system gives a wrong answer, the investigation usually starts at the prompt and works backwards. It should often start at recall: measure what fraction of true nearest neighbours your index returns at your current settings, because if that number is 0.85 then fifteen percent of the time the best chunk never reached the model at all, and no prompt engineering will recover it.

KEY POINTS

- → In RAG the vector store holds chunks and their embeddings for retrieval.
- → Chunk size decides what a single vector must represent.
- → The embedding model defines similarity and cannot change without a full re-index.
- → The distance metric must match how the embedding model was trained.
- → ANN recall is a hard ceiling on the whole application's accuracy.
- → Debug RAG failures at the retrieval layer before touching the prompt.

The full path, and the knob at each stage

Six stages, and five of them are decisions that silently cap what the model can do. Only the last is visible in a prompt, which is why it receives most of the attention and deserves less of it.

```
document ──chunk──> passages ──embed──> vectors ──index──> ANN structure
            |                   |                   |
         size,               which model,      M, ef_search,
         overlap,            which metric      nprobe, recall
         boundaries

question ──embed──> vector ──search──> top-k chunks ──> prompt ──> answer
              |                  |                          |
        same model as       k, filters              the only part
        the corpus                                  most teams tune
```

Measuring the ceiling

Run this before any prompt work. If ANN recall against exact search is below about 0.95 at your production settings, tune the index first — everything downstream is capped by this number and no prompt will lift it.

```
exact = exact_search(query_vecs, matrix, k=10)      # ground truth
approx = index.knn_query(query_vecs, k=10)[0]

recall = np.mean([
    len(set(a) & set(e)) / len(e) for a, e in zip(approx, exact)
])
print(f"ANN recall@10 = {recall:.3f}")

# 0.85 means the best chunk is missing roughly 15% of the time,
# and the model never sees it. that is not a prompting problem.
```

Glossary

Semantic gap

The mismatch between how computers store data — files, fields, tags — and how humans understand its meaning.

Vector embedding

An array of numbers representing an item, positioned so that semantically similar items land close together.

Vector database

A store built to hold embeddings and retrieve them by similarity, with the indexing and filtering machinery that makes that fast.

Cosine similarity

The cosine of the angle between two vectors. Ignores magnitude, which is usually what you want for text.

Euclidean distance

Straight-line distance between two points in the vector space. Sensitive to magnitude as well as direction.

Dot product

A similarity measure combining alignment and magnitude. Equivalent to cosine when vectors are normalised.

L2 normalisation

Scaling a vector to unit length, which makes cosine, dot product and Euclidean produce identical rankings.

Approximate nearest neighbour (ANN)

Algorithms that find vectors very likely to be the closest, trading a little accuracy for a large speed gain.

HNSW

Hierarchical Navigable Small World: a multi-layer proximity graph navigated greedily from sparse upper layers down to the dense base.

IVF

Inverted File Index: partitions the space into clusters and searches only the nearest few, controlled by nprobe.

ef_search

HNSW's query-time beam width. The main dial for trading recall against latency, changeable without rebuilding the index.

nprobe

How many IVF clusters a query searches. One is fastest and least accurate; all of them is equivalent to exhaustive search.

Quantisation

Compressing vectors to fewer bits per dimension to cut memory, at some cost in precision. Often paired with IVF at large scale.

Recall@k

The fraction of true nearest neighbours an approximate index actually returns. The ceiling on any system built on it.

Static vs contextual embedding

Static models give each word one fixed vector; contextual encoders produce representations that depend on the surrounding text.

Check yourself

Answer first, then open each one.

Why can't you close the semantic gap by simply adding more tags to your relational schema?

Because tags are discrete, unary properties chosen in advance, while similarity is continuous and relational. There is no way to express 'somewhat sunset' or 'close to image 42' as a column value, and nobody tags for an attribute until after someone needs it. The limitation is structural, not a matter of effort.

Your embedding model was trained with cosine similarity, but your index is configured for Euclidean distance and the vectors are not normalised. What happens?

Nothing visibly. Queries succeed and results come back ranked, but the ordering is influenced by vector magnitude — which the model never intended to carry meaning — so relevance quietly degrades. The fix is to L2-normalise on insert, after which all three metrics rank identically anyway.

Why is it impossible to build a filter by thresholding a single dimension of a real embedding?

Because meaning is distributed across the whole vector rather than allocated one concept per axis. No coordinate corresponds to 'elevation' or any other nameable property, so a threshold on it selects nothing coherent. Anything you need to filter on must be extracted explicitly and stored as metadata.

What do HNSW's upper layers actually contribute, given that every vector lives in the bottom layer?

They cover long distances in few hops. Starting a greedy walk directly on the dense bottom layer would take many more steps and risks settling in a local minimum far from the query. The sparse upper layers act as an express network that gets the search into the right neighbourhood before the fine-grained work begins.

Why does an IVF index with nprobe=1 miss neighbours that are extremely close to the query?

Because it searches only the single nearest cluster, and a vector sitting just across a cluster boundary is never examined regardless of its actual distance. Cluster membership is decided at index time by proximity to a centroid, not by proximity to any particular query. Raising nprobe searches neighbouring cells and recovers them.

A RAG system gives a wrong answer. Why should you measure ANN recall before rewriting the prompt?

Because recall is a hard ceiling sitting below everything visible. If the index returns 0.85 of true nearest neighbours, then roughly fifteen percent of the time the best chunk never reaches the model at all, and no prompt can compensate for text that was never supplied. Measure against exact search on the same vectors, then work upward.

Where to go next

- → Build an exact search over 100,000 of your own vectors and use it as ground truth, then measure your production index's recall@10 at its current settings — most teams have never seen this number.
- → Sweep ef_search from 16 to 512 and plot recall against p95 latency; the knee of that curve is your actual operating point, and it is rarely where the default sits.
- → Compare HNSW and IVF on the same data at matched recall and compare memory footprint — the gap decides which is affordable at your scale.
- → Take a query with a selective metadata filter and compare post-filtering, over-fetching and native filtered search; the result counts alone are usually enough to settle the design.
- → Re-embed a small corpus with a different model and confirm for yourself that cross-model queries return confident nonsense — the failure is much more memorable once seen.
- → Try product quantisation on a million-vector index and measure the three-way trade between memory, recall and latency.
- → Read the HNSW paper (Malkov & Yashunin, 2016) for why a hierarchy of proximity graphs gives logarithmic-ish search behaviour.
